Progress Report 5

Student Name: Advitiya Sharda

Student ID: 300395470

Project: Smart Door Anomaly Detection & Cybersecurity for Elderly Care

Course: CSIS 4495 – Applied Research Project, Section 3

Reporting Period: March 17 – March 23, 2026

Report Date: March 23, 2026

Work Date / Hours Logs

Date	Hours	Description of Work Done
Mar 17, 2026	1.5	Fixed calibration update target in calibrate_recognition.py and routes.py — script was patching the wrong constant.
Mar 19, 2026	2.0	Added recognize_all_faces() for multi-face frame support. /api/recognize now returns face_count and a faces[] array for all detected faces in a frame.
Mar 20, 2026	2.5	Added _score_face_quality() to evaluate face ROI by size, blur, and brightness before encoding. Wired quality check into register_faces.py to reject dark or blurry registration photos.
Mar 21, 2026	3.0	Implemented RecognitionBuffer class for sliding-window majority-vote smoothing. Added recognize_with_smoothing() method. Wired smoothing into /api/recognize for single-face frames.
Mar 22, 2026	2.0	Rewrote test_face_recognition_real.py: DB person ID resolution via _resolve_person_id(), quality gating, replaced input() with argparse CLI flags, fixed CWD-dependent path.
Mar 23, 2026	2.5	Fixed KeyError: 'user_id' crash in test_facial_recognition.py integration test. Report creation. | Team meeting | Progress Report

Total Hours: 13.5

Summary Description of Work

This week I focused entirely on hardening and extending the facial recognition system — improving accuracy, stability, correctness, and test coverage.

Calibration Fix

Corrected a silent bug where calibrate_recognition.py was writing the optimised threshold t o the wrong target. The script was patching a constant that the live engine never read. Fixed to update DISTANCE_MATCH_THRESHOLD directly in api/facial_recognition.py and verified t he calibrated value is picked up at engine startup.

Recognition Engine Improvements

The confidence scoring denominator bug from last week was extended further: OpenCV mode now uses its own DISTANCE_MATCH_THRESHOLD constant consistently throughout, including in _score_face_quality(). A new quality scoring method was introduced that evaluates each face ROI before registration:

- **Size check** — minimum 50×50 pixels
- **Blur check** — Laplacian variance threshold
- **Brightness check** — rejects overexposed or underexposed faces

Photos that fail any check are skipped with a printed reason, and a per-person summary of skipped photos is shown.

Multi-Face Recognition

The /api/recognize endpoint previously only processed the first detected face in a frame. This week I added recognize_all_faces() which loops over all detected faces, extracts encodings, and returns individual results per face. The endpoint now returns a face_count field and a faces[] array covering every face in the frame.

Temporal Smoothing

To reduce flickering recognitions on video frames, I implemented a RecognitionBuffer class:

- Stores the last N recognition results in a deque
- Uses Counter majority vote to pick a stable person_id
- Averages confidence across the buffer window
- Returns is_stable and sample_count flags so the API consumer knows when a result is reliable

For single-face frames, /api/recognize now calls recognize_with_smoothing(). For multi-face frames, it uses raw results and resets the buffer (since the person composition has changed).

Test Infrastructure Fixes

test_face_recognition_real.py had a critical design flaw: it registered faces using the folder name as the person ID, which never matched the real DB person IDs (e.g., resident_001).

The rewrite adds a _resolve_person_id() helper with three lookup strategies — exact user_id match, title-cased name match, and partial substring match — so test recognition results now align with what the live /api/recognize endpoint would return.

The test_facial_recognition.py integration test had a KeyError crash caused by column name mismatch between the print statement and get_access_logs() return schema. Fixed with .get() fallbacks covering both old and new key names.

Repo Check-in of Implementation

The following files were committed:

- api/facial_recognition.py — _score_face_quality(), RecognitionBuffer, recognize_with_smoothing(), recognize_all_faces()
- api/routes.py — multi-face handling in /api/recognize, single-face smoothing, GET /api/recognition/status endpoint
- scripts/register_faces.py — quality gating integrated into photo registration flow
- tests/test_face_recognition_real.py — full rewrite: DB ID resolution, quality scoring, CLI args, absolute path fix
- tests/test_facial_recognition.py — KeyError fix in integration test print statements

AI Use Section

AI Tool	Version / Account	Feature Used	Value Added by Me
Cursor	claude-4.6-sonnet	Code generation (RecognitionBuffer, multi-face endpoint, quality scoring, test rewrite)	Reviewed all generated code, verified logic flow, adjusted buffer window size and majority-vote tie-breaking, validated DB ID resolution strategies
ChatGPT	Pro Tier	Research (temporal smoothing approaches for real-time face recognition, face quality metrics)	Adapted sliding window concept to deque-based buffer, selected Laplacian variance as blur metric suited to our low-resolution door camera frames

---

Appendix: Prompt History

Prompt 1: "What are common temporal smoothing strategies for stabilizing face recognition results across video frames?"

- Used: Sliding window with majority vote concept
- Modified: Implemented as a deque-based RecognitionBuffer with configurable window size, averaged confidence, and is_stable / sample_count flags for the API consumer

Prompt 2: "What metrics are used to score face image quality before running recognition — size, blur, brightness?"

- Used: Laplacian variance for blur detection, pixel intensity for brightness
- Modified: Combined into a single _score_face_quality() method returning a composite pass/fail with a human-readable reason field; thresholds tuned for our 640×480 door camera resolution

Prompt 3: "How to resolve a folder name to a database record when the naming conventions don't match exactly?"

- Used: Multi-strategy lookup pattern (exact → normalized → partial)
- Modified: Applied as _resolve_person_id() with three ranked strategies specific to our DB schema (user_id and name fields), with a fallback warning printed when no match is found